

ChamVerDB: A Chameleon Hash-Based Secure and Efficient Verifiable Outsourced Database

Wenqi Zhang[✉], Xiong Li[✉], *Senior Member, IEEE*, Ruirui Gao, Ting Chen[✉], *Member, IEEE*,
Jiguo Yu[✉], *Fellow, IEEE*, Jinjun Chen[✉], *Fellow, IEEE*, and Xiaosong Zhang[✉]

Abstract—A verifiable outsourced database is a crucial technology for ensuring the integrity of query results in a cloud-based outsourced database. However, existing schemes struggle to balance efficient query execution with strong data security and query privacy, limiting practical deployment. To address this challenge, we propose ChamVerDB, a Chameleon Hash-based secure and efficient verifiable outsourced database. ChamVerDB restructures the database into a matrix and encrypts each record into three ciphertext segments using a modified BG encryption. For each row, it generates row parity, computes a corresponding Chameleon Hash Value and proof, while also generating individual parities for each record. The ciphertexts and parities are packaged into an encrypted database and outsourced to the server. During querying, the user utilizes a one-hot query vector to retrieve the target ciphertext row and a column vector to extract the desired ciphertext locally. Subsequently, the user and server collaboratively perform proxy re-encryption on one ciphertext segment and recover the plaintext using the remaining segments. The user then verifies the result using row parity, individual parity, proof, and the public hash value. Security analysis confirms that ChamVerDB ensures data security, query privacy, and result verifiability. Experimental evaluation on a 2^{20} -record database demonstrates that ChamVerDB achieves $42\times$ and $32\times$ faster offline processing than two encrypted

outsourcing baselines. In online phase, ChamVerDB completes a query in just 1.21 s, outperforming three representative baselines by $511\times$ (618.5s), $1039\times$ (1258.14s), and $209810\times$ (253871.49 s), illustrating its practical usability.

Index Terms—Verifiable database, Chameleon hash, proxy re-encryption, data security, query privacy.

I. INTRODUCTION

WITH the emergence of the cloud-computing era, data are continuously generated, shared, and exchanged among organizations and individuals, leading to an exponential growth in the data volume that must be stored and maintained. To alleviate the rising costs of local data management, a growing number of enterprises and individuals opt to outsource their data storage to commercial cloud platforms, such as Google Drive, Microsoft Azure, and Amazon Web Services. This outsourcing paradigm provides significant advantages, including reduced operational overhead, enhanced scalability, and high service availability.

Despite these advantages, outsourcing introduces critical security, privacy, and data integrity challenges. Once data are transferred to the cloud, the owner loses direct control and must depend on the reliability of the cloud service provider (CSP). Numerous security incidents have raised concerns about the trustworthiness of CSPs. For instance, a system administrator could intentionally delete portions of the client's data to reduce storage costs, or a malicious insider could modify outsourced data and return incorrect computation results. Moreover, CSPs may analyze user queries to leak sensitive information, compromising query confidentiality. In addition to insider threats, external adversaries can utilize side-channel attacks and other sophisticated techniques to steal confidential data, further exacerbating the security risks inherent in cloud-based storage.

These vulnerabilities highlight the urgent need for robust cryptographic mechanisms, verifiable computing frameworks, and privacy-preserving techniques to protect data integrity, result validity, and user privacy within outsourced environments. Verifiable Outsourced Database (VDB) [1] has become a key cryptographic building block for addressing these issues, allowing data owners to outsource the database to potentially untrusted CSPs while enabling users to verify query correctness efficiently. An effective VDB scheme must satisfy three core security requirements: data security, query privacy, and result verifiability.

Received 27 October 2025; revised 6 May 2026; accepted 3 June 2026. Date of publication 8 June 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U25A20429, Grant 62332018, Grant 62332004, and Grant 62272256, in part by Sichuan Provincial Natural Science Foundation under Grant 2025ZNSFSC0497, in part by Intelligent Systems Security Laboratory under Grant Q25008*2, in part by the Fundamental Research Funds for Chinese Central Universities under Grant ZYGX2025TS010, in part by Shandong Provincial Natural Science Foundation under Grant ZR2022ZD03, and in part by the Colleges and Universities 20 Terms Foundation of Jinan City under Grant 202228093. (Corresponding author: Xiong Li.)

Wenqi Zhang, Xiong Li, and Ruirui Gao are with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: wenqizhang@std.uestc.edu.cn; lixiongzhq@163.com; 202511081823@std.uestc.edu.cn).

Ting Chen is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China, and also with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: brokendragon@uestc.edu.cn).

Jiguo Yu is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China, and also with the Big Data Institute, Qilu University of Technology, Jinan 250353, China (e-mail: jiguoYu@sina.com).

Jinjun Chen is with the Swinburne University of Technology, Melbourne, VIC 3122, Australia, and also with the Muroran Institute of Technology, Muroran 050-0071, Japan (e-mail: jinjun.chen@gmail.com).

Xiaosong Zhang is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China, and also with the Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: johnsonzxs@uestc.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3701191

Existing VDB solutions [2], [3] often involve trade-offs between efficiency and guarantees of data security, query privacy, and result verifiability. Some approaches emphasize verification by utilizing cryptographic primitives such as homomorphic hashing, bilinear pairings, or Merkle trees to detect unauthorized modifications. However, these techniques may result in significant computational costs and communication overhead, thus limiting their suitability for large-scale deployments. Other solutions prioritize query privacy by integrating mechanisms such as Private Information Retrieval (PIR) [4], [5] or Symmetric Searchable Encryption (SSE) [6], [7], [8], [9] to prevent query leakage. Despite these advancements, achieving a unified solution that provides strong security while maintaining low query latency remains a challenging issue.

A. Related Work

Existing research on VDB mainly focuses on two key aspects: verifiability and privacy. Verifiability ensures that users can efficiently verify the integrity of query results, while privacy aims to protect users' query privacy. We review related work from these two perspectives.

Focus on Verifiability: Miao et al. [2] proposed a public VDB scheme supporting comprehensive dynamic operations, including insertion, deletion, and modification, through a hierarchical vector commitment mechanism. Building upon this, Miao et al. [10] developed a static VDB architecture that enables dynamic updates by integrating Merkle Sum Hash Trees (MSHTs) with delegated polynomial functions, thereby achieving both verifiability and updatability. Wu et al. [11] introduced ServeDB, a verifiable framework for secure multidimensional range queries, which employs hierarchical cube mapping and a balanced binary index tree enhanced with Bloom filters to ensure efficient and privacy-preserving queries. Chen et al. [12] proposed the Committed Invertible Bloom Filter (CIBF), which combines invertible Bloom filters with vector commitments, and subsequently introduced an improved version, Increasing CIBF, to enhance indexing performance and query localization. Wang et al. [13] designed a privacy-preserving VDB construction based on polynomial commitments, which guarantees query integrity without information leakage, albeit with high computational cost due to large-scale polynomial interpolation. Zhou et al. [3] presented VeriDB, which utilizes Intel SGX to deliver secure and verifiable database services. Subsequent research has further investigated encrypted VDBs leveraging trusted execution environments (TEEs) [14], [15], [16]. Li et al. [17] proposed PVPKEET, a public key encryption scheme supporting public verifiability and equality testing, enabling integrity verification without decryption. Yang et al. [18] designed VUSDB, a matrix commitment-based verifiable database scheme. Guan et al. [19] designed a framework for fair outsourced polynomial computation, though it exhibited privacy leakage and a non-trivial false positive rate. To mitigate these issues, Wang et al. [20] integrated homomorphic encryption, blind signatures, and TEEs to realize privacy-preserving verifiable computation with a reduced false positive rate.

Focus on Privacy: Beyond ensuring result verifiability, preserving user query privacy remains a fundamental challenge

in VDB. This concern has spurred significant advances in Private Information Retrieval (PIR), which has found applications beyond databases, including certificate transparency logs [21], anonymous web search [22], anonymous messaging [23], [24], vulnerability disclosure [25], and private media access [26]. To improve PIR efficiency, Ghoshal et al. [27] and Kogan et al. [28] employed puncturable pseudorandom functions (PRFs) for efficient query evaluation, while Bienstock et al. [5] introduced a batch PIR scheme based on oblivious ciphertext aggregation and compression. Lazzaretti et al. [29] further optimized Kogan et al.'s construction by reducing computational and communication overhead. Despite these advancements, PIR schemes often face a trade-off between strong privacy protection and computational efficiency [30]. To address both privacy and verifiability, Colombo et al. [31] proposed a verifiable PIR scheme based on the Decisional Diffie-Hellman (DDH) and Learning with Errors (LWE) assumptions. While offering robust privacy and integrity, their DDH-based instantiation incurs notable computational overhead due to numerous modular exponentiations. In many PIR designs, integrity checks are directly integrated within the retrieval or reconstruction phases [28], [29]. In parallel, Searchable Encryption (SE), particularly Symmetric Searchable Encryption (SSE), has been extensively studied for secure query processing. SSE enables efficient keyword search over encrypted data while preserving both data confidentiality and query privacy across diverse application scenarios [6], [7], [8], [9], [32], [33].

B. Motivation

Despite significant progress in VDB research, existing solutions rarely achieve efficiency while simultaneously ensuring security, privacy, and verifiability. This work aims to design a scheme that achieves the following objectives: (1) Data Security: Ensure the security of the data owner's data without relying on a trusted third party (TTP) or the continuous involvement of the data owner. (2) Query Privacy and Integrity: Protect users' query privacy while enabling integrity verification of the obtained query results. (3) Efficiency: Achieve practical and efficient query processing and integrity verification under the constraints of security and privacy protection.

C. Our Contributions

To address the above challenges, we propose ChamVerDB, a Chameleon hash-based secure and efficient Verifiable DataBase. The main contributions of this work are as follows:

- 1) We propose *mBG*, a modified BG (Brakerski-Goldwasser) encryption scheme that encrypts data into three segments. It supports plaintext recovery via proxy re-encryption using ciphertexts derived from queries. Additionally, we design a lightweight RSA-based chameleon hash (*CH*) that enables both verifiable query results and efficient data updates.
- 2) Building on *mBG* and *CH*, we construct ChamVerDB, which restructures the database into a matrix and utilizes *mBG* to ensure data security. During querying, it retrieves the target ciphertext row using a query vector and then the exact ciphertext locally using another query

vector, ensuring query privacy. In addition, it utilizes proxy re-encryption for plaintext recovery and CH to enable efficient result verification and data updates.

- 3) Experimental results demonstrate that ChamVerDB achieves high computational efficiency with acceptable communication overhead. For a database of size 2^{20} , ChamVerDB achieves a $42\times$ and $32\times$ improvement in offline outsourcing efficiency compared to encrypted outsourcing baselines, and an online query latency of only 1.21 s, representing speedups of $209810\times$, $1039\times$ and $511\times$ over the existing solutions.

The remainder of this paper is organized as follows. Section II introduces the preliminaries used in ChamVerDB. Section III outlines the system models, design goals and application scenarios. Section IV describes the high-level overview of ChamVerDB. Section V presents our mBG and CH schemes, followed by a detailed description of ChamVerDB. Section VI provides the correctness and security analysis. Section VII evaluates and compares the performance. Finally, Section VIII concludes the paper.

II. PRELIMINARY

In this section, we introduce some cryptographic preliminaries used in our ChamVerDB.

A. Verifiable Database

A VDB scheme [2] typically involves three entities: Data Owner ($\mathcal{DO}$), Cloud Server ($\mathcal{CS}$), and Query Users ($\mathcal{QU}$). It comprises the following algorithms:

Setup(λ, DB, n): Performed by $\mathcal{DO}$, this algorithm takes as inputs the security parameter λ and the database DB with size n . It outputs a public-private key pair (pk, sk) , a set of public parameters PP , and a processed database DB .

Query(PP, DB, i): This deterministic algorithm, operated by $\mathcal{CS}$, processes a query for index i under PP and returns the data item x_i in DB , along with its corresponding proof π .

Verify(PP, x_i, π): This algorithm, executed by $\mathcal{QU}$, takes as input the PP and the response (x_i, π) . It returns 1 if the returned data is verified and correct; otherwise, it outputs $\perp$ to signify a verification failure.

Update(sk, i, x'): This algorithm, jointly performed by $\mathcal{DO}$ and $\mathcal{CS}$, uses the secret key sk , target index i , and updated data x'_i as input to carry out the database update operation.

B. Chameleon Hash

The Chameleon Hash (CH) [34] is a special cryptographic hash function that retains conventional properties such as one-wayness and collision resistance while incorporating a “trapdoor” mechanism. This mechanism enables the holder of a secret trapdoor key to efficiently generate collisions, i.e., modify a message without changing its hash output. Although this relaxes standard collision resistance, it provides controlled malleability that is useful in various cryptographic constructions.

Chameleon hash functions are typically employed in applications that require verifiability and controlled mutability, such as

redactable blockchains [35]. In a redactable blockchain, the hash of each block is computed using a chameleon hash, allowing an authorized party with the trapdoor key to modify block content by generating new randomness that retains the original hash value, thus avoiding disruption of subsequent blocks. Motivated by this property, we extend chameleon hashes to verifiable databases, enabling the data owner to update encrypted entries while preserving existing hash validity, which significantly reduces recomputation overhead and enhances update efficiency.

A Chameleon Hash consists of five probabilistic polynomial-time ($\mathcal{PPT}$) algorithms, such that:

$PP \leftarrow CH.PG(1^\lambda)$: Given a security parameter 1^λ , the parameter generation algorithm outputs the public parameter PP for the chameleon hash.

$(pk, sk) \leftarrow CH.KG(PP)$: Given the public parameters pp as input, the algorithm outputs the private key sk and public key pk for the chameleon hash.

$(h, \pi) \leftarrow CH.Hash(pk, m)$: Given the public key pk and a message m , this algorithm selects a randomness π and computes a corresponding hash h , outputs the pair (h, π) .

$\{0, 1\} \leftarrow CH.Check(pk, m, \pi, h)$: Given the public key pk , message m , randomness π , and hash value h , the algorithm outputs a bit $d \in \{0, 1\}$ indicating the validity of the hash h .

$\pi' \leftarrow CH.Adapt(sk, m, m', \pi, h)$: Given the private key sk , message m , new message m' , randomness π , and h , the algorithm outputs a new randomness π' to update m to m' .

Generally, a chameleon hash possesses three key properties: collision-resistance [34], indistinguishability [34], and uniqueness [36]:

Definition 1 (Collision Resistance): A chameleon hash function provides collision-resistance if, given a message m and the corresponding chameleon hash pair (h, π) , no $\mathcal{PPT}$ adversary $\mathcal{A}$, without knowledge of the trapdoor (i.e., the secret key sk), can produce a different message m' and a corresponding randomness π' such that $CH.Check(pk, m, \pi, h) = CH.Check(pk, m', \pi', h)$.

Definition 2 (Indistinguishability): A chameleon hash function provides indistinguishability if no $\mathcal{PPT}$ adversary $\mathcal{A}$ can determine whether a given randomness (specifically, the proof π in our construction) was produced by $CH.Hash$ or derived via $CH.Adapt$.

Definition 3 (Uniqueness): A chameleon hash function is said to satisfy uniqueness if, for a given public key pk and any message m , once the randomness π in $CH.Hash$ is chosen, the hash value h of $CH.Hash(pk, m)$ is uniquely determined.

C. Bilinear Map

Bilinear map was originally proposed as a cryptographic primitive in an identity-based encryption scheme [37]. Formally, let $\mathbb{G}$ and $\mathbb{G}_T$ be multiplicative cyclic groups of large prime order p , and g be a generator of $\mathbb{G}$. A map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ is called bilinear if it satisfies the following properties:

- *Bilinear*: For all $u, v \in \mathbb{G}$, and $a, b \in \mathbb{Z}_p$, it holds that $e(u^a, v^b) = e(u, v)^{ab}$.
- *Non-degenerate*: There exists $u, v \in \mathbb{G}$ such that $e(u, v) \neq 1$.

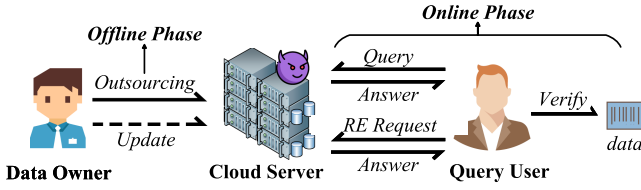


Fig. 1. System Model of ChamVerDB.

- *Computable*: There exists a polynomial time algorithm to compute $e(u, v)$ for all $u, v \in \mathbb{G}$.

Definition 4 (Decisional Diffie-Hellman (DDH) Assumption [38]): Let $\mathbb{G}$ be a cyclic group of large prime order p with generator g . Given two tuples (g, g^x, g^y, g^{xy}) and (g, g^x, g^y, g^z) , where $x, y, z \in \mathbb{Z}_p$ are chosen uniformly at random, no $\mathcal{PPT}$ adversary can distinguish between them with non-negligible advantage.

III. SYSTEM DESIGN

In this section, we introduce the system and threat models, the design goals as well as the application scenarios of our ChamVerDB.

A. System Model

The system model of ChamVerDB is similar to that of PIR schemes [21], [22], [26], [31], while further extending it by incorporating security guarantees for the database. As illustrated in Fig. 1, the system model comprises three entities: the data owner (DO), the cloud server (CS), and the query user (QU). The system operates in two distinct stages: the offline phase and the online query phase. In the offline phase, DO structures the database into a matrix format, computes row-level proofs, encrypts the matrix, and outsources it to CS. Simultaneously, DO generates re-encryption keys corresponding to each QU. During the online query phase, upon receiving a query vector from QU, CS returns the corresponding encrypted row. QU then extracts the target ciphertext and initiates a re-encryption request. In response, CS generates a re-encryption ciphertext for QU. Finally, QU recovers the plaintext and verifies its integrity using the chameleon hash.

Similar to PIR schemes, the querier QU in ChamVerDB is assumed to know the index of the cloud-stored database, enabling the construction of query vectors accordingly. Simple multi-table queries (e.g., those using index-linked primary keys) can be handled through multiple executions, while the more complex queries are discussed in Section VIII.

B. Threat Model

In our ChamVerDB, we assume that DO is fully trusted, QU is honest-but-curious (HBC), and CS is malicious. Specifically, a malicious CS may return falsified data instead of the authentic records provided by DO, and may also attempt to infer the exact data index targeted by QU's query. An HBC QU may exploit the batch ciphertext returned by CS to derive information beyond the legitimate query result.

C. Design Goals of ChamVerDB

ChamVerDB is designed to fulfill the following goals:

- 1) *Data security*: The original data of DO remains unrecoverable by CS from the outsourced encrypted database EDB. Specifically, for any index x_0, x_1 , no $\mathcal{PPT}$ CS, can distinguish $EDB(x_0)$ from $EDB(x_1)$ with non-negligible advantage.
- 2) *Query Privacy*: The query request issued by QU preserves privacy against other entities. Formally, for any two queries $Q(i, j)$ and $Q(i', j')$ over $\{EDB_{ij}\}_{i=1, j=1}^n$, no $\mathcal{PPT}$ adversary, including CS, can distinguish between them with non-negligible advantage.
- 3) *Verifiability*: QU can verify the validity of the retrieved data $x_i \in DB$ from the query results.
- 4) *Efficiency*: The scheme must operate efficiently, with all computations, particularly online queries, completed within polynomial time.

D. Application Scenarios

Since the system model of ChamVerDB closely aligns with that of PIR schemes [21], [22], [26], [31], it is particularly suitable for privacy-preserving retrieval of fixed-length indexed records, while additionally ensuring dataset security and result verifiability. A representative 32-bit application is network security retrieval over IPv4-indexed databases. In such settings, organizations, i.e., DO, may outsource threat intelligence databases to the cloud, where each entry corresponds to a 32-bit IPv4 address or a compact security value associated with that address, such as a blacklist indicator, risk label, routing policy code, or audit tag. A QU can then query the detail of one target IPv4-related entry, without revealing the queried item to the CS, while still being able to verify the correctness of the returned result. Beyond this scenario, ChamVerDB is also applicable to other outsourced databases of compact records, such as metadata entries in cloud storage, short audit or anomaly labels in financial and enterprise systems, and lightweight indexed tags in distributed learning or edge computing platforms.

IV. TECHNICAL OVERVIEW

In this section, we present a technical overview of ChamVerDB and take the IPv4 blacklist check as an example. In such a system, an organization (DO) maintains a large database DB containing security-sensitive indexed data, such as IPv4 addresses in a blacklist, and outsources it to a cloud server (CS) for storage and query services. Authorized users (QUs), such as analysts, administrators, or security operators, issue queries to retrieve a specific IPv4 address. In this setting, several critical challenges arise: (1) the system must support highly efficient large-scale queries, (2) CS should not learn the outsourced sensitive IPv4 address (data security), (3) CS should not infer which IPv4 address is being queried (query privacy), and (4) the returned IPv4 address must be correct and untampered (verifiability). ChamVerDB is designed to address these challenges in an integrated manner. At a high level, ChamVerDB combines matrix-based retrieval to reduce online query cost and hide the exact query index, *mBG* to protect outsourced data, proxy

re-encryption to enable authorized recovery without exposing secret keys to $\mathcal{CS}$, and row-level chameleon-hash-based verification to detect forged results with low overhead. Next, we will give the point-to-point overview

To achieve high efficiency in large-scale data queries, ChamVerDB adopts a space-for-time trade-off strategy. It restructures a database of size n^2 into an $n \times (n+1)$ matrix by adding an additional proof column. Each query retrieves an entire row containing $n+1$ ciphertexts, revealing only the row index i . The user then locally extracts the target j -th ciphertext. This design significantly reduces server-side computation latency, which is crucial in time-sensitive applications, while preserving query privacy through matrix-based access patterns similar to prior PIR schemes such as SimplePIR [21].

To protect sensitive data from an untrusted $\mathcal{CS}$, ChamVerDB ensures data security through encryption before outsourcing. A key requirement in outsourced queries is that only authorized $\mathcal{QUs}$ can get specific data without exposing additional information. To achieve this, ChamVerDB introduces a modified BG encryption scheme (mBG) with proxy re-encryption. Each data is encrypted into a ciphertext tuple (c_1, c_2, c_3) , where c_1 protects the data content, c_2 conceals encryption randomness, and c_3 enables controlled re-encryption. During a query, $\mathcal{QU}$ leverages this structure to securely recover the required randomness and decrypt only the requested data. In practical terms, this mechanism allows $\mathcal{CS}$ to process the query without learning either the plaintext data or the decryption capability of $\mathcal{QU}$. This is especially important in outsourced security services, where storage and computation may be delegated, but trust cannot be fully delegated.

To prevent leakage of sensitive user intent, ChamVerDB decomposes each query into a row vector q_1 and a column vector q_2 . $\mathcal{QU}$ sends q_1 to $\mathcal{CS}$ to retrieve an encrypted row, while q_2 is used locally to extract the desired ciphertext. This design makes it difficult for $\mathcal{CS}$ to infer the actual query index, which is essential in privacy-sensitive security applications. Even if $\mathcal{CS}$ receives repeated query requests, it can only learn limited row-level information rather than the precise index. This directly mitigates the risk of leaking the user's intent, such as which IPv4 address or blacklist entry is under examination.

To ensure the correctness and integrity of returned data, ChamVerDB provides verifiability through a lightweight Chameleon hash (CH) construction. For each row, a parity value p_i , hash h_i , and proof π_i are generated. Upon receiving query results, $\mathcal{QU}$ can verify both the correctness of the returned row and the integrity of the extracted data. This design supports efficient batch verification and minimizes verification overhead. Furthermore, when a data is updated, only the corresponding row proof needs to be refreshed, enabling efficient database maintenance in dynamic outsourced systems. This verification capability is especially valuable in dependability-critical settings, because it allows $\mathcal{QUs}$ to detect tampered or inconsistent query results returned by an untrusted $\mathcal{CS}$ without reprocessing the entire database.

Overall, by combining matrix-based querying, proxy re-encryption, and lightweight verifiable structures, ChamVerDB

achieves secure, private, and verifiable query processing with high efficiency. It avoids expensive cryptographic operations such as large-scale homomorphic computation, requiring only a small number of modular exponentiations for both query processing and verification, making it well-suited for practical large-scale data query services.

V. OUR CONSTRUCTION: CHAMVERDB

This section first introduces our improved BG encryption mBG and Chameleon hash CH , followed by a formal description of the proposed ChamVerDB built upon them.

A. The Construction of the mBG

We introduce mBG , a modified encryption scheme derived from the Brakerski-Goldwasser encryption [39], [40], specifically modified for query scenarios while maintaining security guarantees. mBG involves three key entities: the Sender ($\mathcal{S}$), the Receiver ($\mathcal{R}$) and the Administrator ($\mathcal{AD}$). The scheme proceeds as follows:

1) $(PP, pk, sk) \leftarrow mBG.Setup(1^\lambda, l)$: Given the security parameter 1^λ and integer l , $\mathcal{AD}$ selects three large prime numbers p , q_1 and q_2 , and computes $N = q_1 q_2$. It then defines a hash function $H_2 : \mathbb{G}_T \rightarrow \mathbb{Z}_{N^2}^*$ and a bilinear mapping $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$, where $\mathbb{G}$ and $\mathbb{G}_T$ are two multiplicative cyclic groups of order p . Next, $\mathcal{AD}$ chooses l random elements $\{g_1, g_2, \dots, g_l\} \in \mathbb{Z}_{N^2}^*$, selects $\hat{g} \in \mathbb{G}$ and $\hat{g}_t \in \mathbb{G}_T$, and computes $Z = e(\hat{g}, \hat{g}) \in \mathbb{G}_T$. The public parameters are then defined as $PP = \{e, \mathbb{G}, \mathbb{G}_T, p, Z, \hat{g}, \hat{g}_t, l, N, H_2, \{g_1, g_2, \dots, g_l\}\}$. To generate a key pair, $\mathcal{R}$ samples an l -bit integer d such that $d \in \mathbb{Z}_p^*$, parses it into a binary vector $d = (d_1, d_2, \dots, d_l) \in \{0, 1\}^l$, and computes $g_0 = \prod_{i=1}^l g_i^{d_i} \bmod N^2$ and $\hat{g}^d$. Finally, $\mathcal{R}$ sets the secret key as $sk = d$ and the public key as $pk = (g_0, \hat{g}^d)$.

2) $ct \leftarrow mBG.Enc(pk, m, PP)$: Given the public parameters PP , a plaintext $m \in \mathbb{Z}_N$ and $\mathcal{R}$'s public key $pk = (g_0, \hat{g}^d)$, $\mathcal{S}$ randomly selects $r, k \in \mathbb{Z}_N^*$ and computes the ciphertext components as $c_1 = (g_0)^{-1} H_2(\hat{g}_t^r) (1 + N)^m \bmod N^2$, $c_2 = \hat{g}_t^r Z^k$ and $c_3 = (\hat{g}^d)^k$. The final ciphertext is then defined as $ct = (c_1, c_2, c_3)$.

3) $m \leftarrow mBG.Dec(sk, ct, PP)$: Given the ciphertext $ct = (c_1, c_2, c_3)$ and the corresponding secret key $sk = d$, $\mathcal{R}$ first computes $\hat{g}_t^r = \frac{c_2}{e(c_3^{\frac{1}{d}}, \hat{g})}$. Subsequently, $\mathcal{R}$ recovers the plaintext by computing $m = \frac{(c_1 \cdot g_0 \cdot H_2(\hat{g}_t^r)^{-1}) - 1}{N} \bmod N$.

B. The Construction of the Chameleon Hash (CH)

The Chameleon hash (CH) enables verifiable retrieval with low parameter overhead and computational cost. Built within the mBG framework, it leverages an RSA-based secret key to ensure the secure and valid proof modifications. The protocol involves two entities, i.e., the Prover ($\mathcal{P}$) and the Verifier ($\mathcal{V}$), and proceeds as follows:

1) $(PP, pk, sk) \leftarrow CH.Setup(1^\lambda)$: Given the security parameter 1^λ , $\mathcal{P}$ selects two prime numbers q_1 and q_2 as in the mBG setup, and computes the RSA modulus $N = q_1 q_2$.

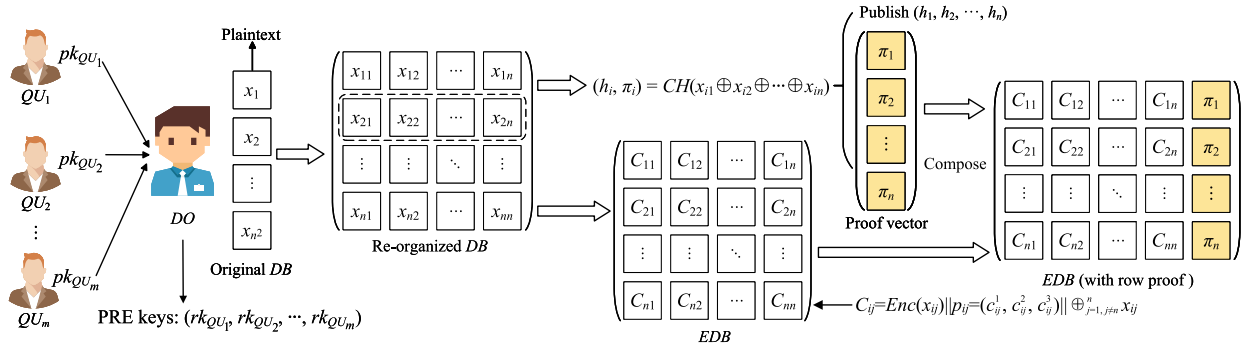


Fig. 2. The offline database outsourcing procedures of ChamVerDB.

$\mathcal{P}$ then chooses $e_{ch} \in (1, N)$ such that $(e_{ch}, \varphi(N)) = 1$, and computes d_{ch} satisfying $d_{ch}e_{ch} \equiv 1 \pmod{\varphi(N)}$.

Next, $\mathcal{P}$ sets $g = N + 1$ and selects a hash function $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_N^*$. Finally, $\mathcal{P}$ publishes the public parameter $PP = \{N, g, H_1\}$, sets $pk = e_{ch}$ as the public key, and retains $sk_{ch} = d_{ch}$ as the secret key.

- 2) $(h, \pi) \leftarrow CH.Hash(PP, pk, m)$: Given the public parameter PP , $\mathcal{P}$'s public key $pk = e_{ch}$ and a message m , $\mathcal{P}$ samples a random $\pi \in \mathbb{Z}_N^*$, and computes $h = g^{H_1(m)} \cdot g^{\pi e_{ch}} \pmod{N^2}$. It then outputs the proof pair (h, π) .
- 3) $\{0, 1\} \leftarrow CH.Check(PP, pk, m, h, \pi)$: Given the public parameter PP , $\mathcal{P}$'s public key $pk = e_{ch}$, a message m and the proof pair (h, π) , $\mathcal{V}$ computes $h' = g^{H_1(m)} \cdot g^{\pi e_{ch}} \pmod{N^2}$. The algorithm then outputs 1 if $h' = h$, indicating successful verification; otherwise, it outputs 0 to signal failure.
- 4) $(\perp, \pi') \leftarrow CH.Adapt(PP, pk, sk, m, m', (h, \pi))$: Given the public parameters PP , $\mathcal{P}$'s public key $pk = e_{ch}$, $\mathcal{P}$'s secret key $sk = d_{ch}$, a message m , a new message m' , and the proof pair (h, π) , $\mathcal{P}$ first invokes $CH.Check$ to verify the validity of h and π . If the check fails, it outputs $\perp$. Otherwise, it computes $\pi' = (H_1(m) + \pi e_{ch} - H_1(m'))^{d_{ch}} \pmod{N}$ and returns the adapted proof pair (h, π') corresponding to m' .

C. Our Verifiable Database ChamVerDB

ChamVerDB operates in three distinct phases: offline, online, and update. The offline phase comprises three steps: Setup, Preprocess, and Outsource, while the online phase consists of Query, Answer, Recon (involving re-encryption requests and responses, along with data decryption), and Verify. For clarity, we assume the database contains n^2 records, with indexing starting from 1. We present a detailed description of ChamVerDB based on its three-phase structure.

1) *The Offline Phase*: In this phase, $\mathcal{DO}$ initializes the system, arranges and encrypts the database, and then outsources it to $\mathcal{CS}$. The workflow of this phase is shown in Fig. 2, and the details are as follows.

Step 1. $(pk, sk, PP, pk_{ch}, sk_{ch}, rk) \leftarrow Setup(1^\lambda, l)$: Given the security parameter 1^λ , $\mathcal{DO}$ generates the public parameters

$PP = \{e, \mathbb{G}, \mathbb{G}_T, p, Z, \hat{g}, \hat{g}_t, l, g, N, H, H_1, H_2, \{g_1, \dots, g_l\}\}$ according to the setup algorithms of mBG and CH , where $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ is a bilinear pairing, $\mathbb{G}$ and $\mathbb{G}_T$ are two multiplicative cyclic groups of large prime order p , $\hat{g} \in \mathbb{G}$ and $\hat{g}_t \in \mathbb{G}_T$, $Z = e(\hat{g}, \hat{g}) \in \mathbb{G}_T$, $N = q_1 q_2$, $e_{ch} \in \mathbb{Z}_N$, $\gcd(e_{ch}, \phi(N)) = 1$, $d_{ch}e_{ch} \equiv 1 \pmod{\varphi(N)}$, $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_N^*$, $H_2 : \mathbb{G}_T \rightarrow \mathbb{Z}_{N^2}^*$, and H are three hash functions, and $\{g_1, \dots, g_l\}$ are l random elements in $\mathbb{Z}_{N^2}^*$ with $l \in [\log N, N]$. Next, $\mathcal{DO}$ chooses an l -bit element $a \in \mathbb{Z}_p^*$ as the secret key, parses it into a binary vector $a = (a_1, \dots, a_l) \in \{0, 1\}^l$, and computes $\hat{g}^a$ and $g_0 = \prod_{i=1}^l g_i^{a_i}$. Then, $\mathcal{DO}$ sets the mBG key pair as $(pk, sk) = ((g_0, \hat{g}^a), a)$, and the CH key pair as $(pk_{ch}, sk_{ch}) = (e_{ch}, d_{ch})$. Similarly, $\mathcal{QU}$ randomly chooses an l -bit element $b \in \mathbb{Z}_p^*$, and sets its key pair as $(pk, sk) = (\hat{g}^b, b)$. At the same time, $\mathcal{DO}$ computes a re-encryption key $rk_{\mathcal{DO} \rightarrow \mathcal{QU}} = (\hat{g}^b)^{\frac{1}{a}}$ for each $\mathcal{QU}$ according to $pk = \hat{g}^b$, and securely sends it to $\mathcal{CS}$.

Step 2. $DB_{n \times (n+1)} \leftarrow Preprocess(DB, PP)$: Given the public parameters PP and a database $DB = (x_1, x_2, \dots, x_{n^2})$, $\mathcal{DO}$ reorganizes DB into an $n \times n$ matrix $DB_{n \times n}$, where each row contains n entries. Here, we assume the database size to be n^2 by default. In fact, the database can be of any dimension. Our aim is to reconstitute the database into row $\times$ column and zeros or random elements can be added at the end of the database if not meet the reorganize principle. In particular, each data item x_m ($m \in [1, n^2]$) is mapped to position (i, j) in $DB_{n \times n}$, where $i = \lceil \frac{m}{n} \rceil$ and $j = m - \lceil \frac{m}{n} \rceil \cdot n$. For each row i , $\mathcal{DO}$ computes a row-level parity value $p_i = \bigoplus_{j=1}^n x_{ij}$, and then computes the individual parity value for each x_{ij} as $p_{ij} = p_i \oplus x_{ij} \oplus H(x_{ij} || j)$ for each x_{ij} , $j \in [1, n]$. The values $(p_1, \dots, p_n)$ are kept locally by $\mathcal{DO}$. Next, for each p_i , $\mathcal{DO}$ generates a chameleon hash h_i along with its corresponding proof π_i using the function $CH.Hash(PP, pk_{ch}, p_i)$. More specifically, $\mathcal{DO}$ randomly chooses $\pi_i \in \mathbb{Z}_N^*$ and computes $h_i = g^{H_1(p_i)} \cdot g^{\pi_i e_{ch}} \pmod{N^2}$. To enable efficient and verifiable queries, the hash values $(h_1, h_2, \dots, h_n)$ are made public, while the proofs $(\pi_1, \pi_2, \dots, \pi_n)$ are attached to the matrix as the $(n+1)$ -th column of $DB_{n \times n}$, forming $DB_{n \times (n+1)}$.

Step 3. $EDB \leftarrow Outsource(DB_{n \times (n+1)}, PP)$: Given PP and $DB_{n \times (n+1)}$, for each data entry x_{ij} where $i, j \in [1, n]$, $\mathcal{DO}$ randomly chooses $r, k \in \mathbb{Z}_N^*$, and computes the

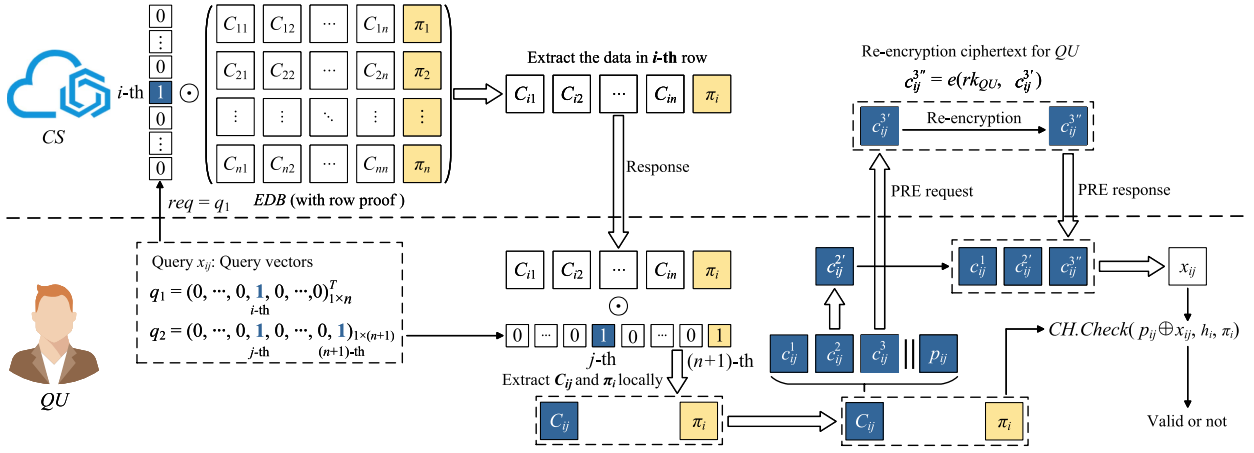


Fig. 3. The online query procedures of ChamVerDB.

ciphertext $c_{ij} = (c_{ij}^1, c_{ij}^2, c_{ij}^3)$, where $c_{ij}^1 = (g_0)^{-1} H_2(\hat{g}_t^r)(1 + N)^{x_{ij}} \bmod N^2$, $c_{ij}^2 = \hat{g}_t^r Z^k$, and $c_{ij}^3 = (\hat{g}^a)^k$. The ciphertext is then concatenated with the corresponding parity p_{ij} to form the encrypted entry $EDB_{ij} = c_{ij} \| p_{ij}$, which is placed at position (i, j) in EDB . The last column of EDB is a copy of DB 's. Finally, DO outsources EDB to CS .

2) *The Online Phase*: In this phase, QU initiates an online query to CS and verifies the validity of the result. The details are shown in Fig. 3 and described as follows.

Step 1. $(q_1, q_2) \leftarrow \text{Query}(m)$: To retrieve x_m from the database, where $m = (i - 1) \times n + j$ corresponds to the (i, j) -th entry in EDB , QU constructs two query vectors, q_1 and q_2 . $q_1 \in \mathbb{F}_2^n$ is a one-hot vector with value 1 at position i and 0 elsewhere, i.e., $(0, \dots, 0, \underset{i\text{-th}}{1}, 0, \dots, 0)_{1 \times n}$, which identifies the target row EDB_i . Meanwhile, $q_2 \in \mathbb{F}_2^{n+1}$ is formed with 1 at positions j and $n + 1$ and 0 elsewhere, i.e., $q_2 = (0, \dots, \underset{j\text{-th}}{1}, \dots, 0, \dots, \underset{(n+1)\text{-th}}{1})_{1 \times (n+1)}$, allowing QU to retrieve both the ciphertext EDB_{ij} and the corresponding proof π_i simultaneously. Finally, QU sends the query request $req = q_1$ to CS and retains q_2 locally for decryption.

Step 2. $EDB_i \leftarrow \text{Answer}(req)$: When receiving req , CS interprets the database as a column vector and computes its Hadamard product with req to derive EDB_i . Then, CS transmits the query result $ans = EDB_i$ to QU .

Step 3. $x_{ij} \leftarrow \text{Recon}(ans)$: Upon receiving the response ans , QU computes the Hadamard product $ans \odot q_2$ and interprets the result as (EDB_{ij}, π_i) . It then extracts $c_{ij} = (c_{ij}^1, c_{ij}^2, c_{ij}^3)$ and p_{ij} from EDB_{ij} . To obtain the plaintext, QU requests a re-encryption from CS . To prevent the PRE request from leaking the information of column j , QU randomly selects $\alpha \in \mathbb{Z}_N^*$, computes $c_{ij}^{3'} = c_{ij}^3 \cdot (\hat{g}^a)^\alpha$, and sends it to CS . Then, CS computes the re-encrypted ciphertext $c_{ij}^{3''} = e(rk_{DO \rightarrow QU}, c_{ij}^{3'}) = e(\hat{g}^{\frac{b}{a}}, \hat{g}^{a(k+\alpha)}) = Z^{b(k+\alpha)}$ and returns it. Next, QU computes $c_{ij}^{2'} = c_{ij}^2 \cdot Z^\alpha$ and decrypts the ciphertext using its secret key b to recover $\hat{g}_t^r = \frac{c_{ij}^{2'}}{(c_{ij}^{3''})^{\frac{1}{b}}} = \frac{c_{ij}^2 \cdot Z^\alpha}{Z^{b(k+\alpha)}^{\frac{1}{b}}}$. Finally, QU executes the decryption procedure as in *mBG*,

computing $\frac{(g_0 H_2(\hat{g}_t^r)^{-1} c_{ij}^1) - 1}{N} \bmod N$ to recover the plaintext x_{ij} .

Step 4. $\{1, 0\} \leftarrow \text{Verify}(x_{ij}, h_i, \pi_i)$: To verify the validity of x_{ij} , QU first reconstructs the row parity $p_i = x_{ij} \oplus p_{ij} \oplus H(x_{ij} \| j)$ using p_{ij} . Then, it executes $CH.Check(PP, pk_{ch}, p_i, h_i, \pi_i)$ by computing $h'_i = g^{H_1(p_i)} \cdot g^{\pi_i^{pk_{ch}}} \bmod N^2$ and checking whether $h'_i = h_i$. If they are equal, x_{ij} is valid; otherwise, it is invalid.

3) *The Update Phase*: This phase enables the DO to update the data stored within the CS .

$EDB^{new} \leftarrow \text{Update}(EDB, i, j, x_{ij}^{new})$: Suppose DO intends to update x_{ij} to a new value x_{ij}^{new} . To maintain data consistency and verifiability, DO must update the ciphertext of x_{ij} , the row parity p_i , the corresponding parity value p_{ij} , and the associated proof π_i .

Specifically, DO first computes the updated row parity $p_i^{new} = p_i \oplus x_{ij} \oplus x_{ij}^{new}$ using the locally stored p_i , and generates a new proof π_i^{new} by invoking $CH.Adapt(PP, pk_{ch}, sk_{ch}, p_i, p_i^{new}, (h_i, \pi_i))$, i.e., $\pi_i^{new} = (H_1(p_i) + \pi_i^{pk_{ch}} - H_1(p_i^{new}))^{sk_{ch}} \bmod N$. Next, DO constructs an n -dimensional parity update vector PV , where the j -th element captures the parity change introduced by x_{ij}^{new} , that is:

$$PV = \begin{cases} H(x_{ij} \| j) \oplus H(x_{ij}^{new} \| j), & j\text{-th element,} \\ x_{ij} \oplus x_{ij}^{new}, & \text{others.} \end{cases}$$

Then, DO randomly selects $k_j \in \mathbb{Z}_N^*$ and computes the updated ciphertext tuple $upt = ((1 + N)^{x_{ij}^{new} - x_{ij}}, Z^{k_j}, (\hat{g}^a)^{k_j})$. Finally, DO sends the update request $(i, j, PV, upt, \pi_i^{new})$ to CS .

Upon receiving the update request, CS parses the ciphertexts of the i -th row into a vector $C = \{c_{ij}\}_{j \in [1, n]} = \{(c_{ij}^1, c_{ij}^2, c_{ij}^3)\}_{j \in [1, n]}$, and the corresponding parity values into a vector $P = \{p_{ij}\}_{j \in [1, n]}$. To update the parities, CS performs element-wise XOR operations: $P_j = P_j \oplus PV_j$. Meanwhile, to update the target entry x_{ij} , CS multiplies the j -th ciphertext tuple with the received upt , computing $c_{ij} =$

$(c_{ij}^1 \cdot (1+N)^{x_{ij}^{new}-x_{ij}}, c_{ij}^2 \cdot Z^{k_j}, c_{ij}^3 \cdot (\hat{g}^a)^{k_j})$. Finally, the updated encrypted entries are assembled as $\{EDB_{ij}^{new}\}_{j \in [1,n]} = \{C_j \| P_j\}_{j \in [1,n]}$, and the updated row proof is set as $EDB_{i(n+1)}^{new} = \pi_i^{new}$, thus completing the update.

Our update strategy is a trade-off between privacy and efficiency. In the update phase, the updated location (i, j) is revealed to $\mathcal{CS}$, so that ChamVerDB can avoid re-encrypting the whole database or recomputing proofs for all rows, thereby greatly reducing the update cost. Meanwhile, the updated content itself remains protected, because $\mathcal{CS}$ only receives masked update information rather than the plaintext.

In addition, ChamVerDB natively supports limited batch updates. When multiple entries in the same row need to be updated, these updates can be handled together. Specifically, $\mathcal{QU}$ can construct one PV for all updated items in the specific row and send the corresponding update tuples together, which reduces repeated row-level cost. For updates involving multiple rows, ChamVerDB handles them by executing the update procedure multiple times. However, we do not provide a dedicated batch-update protocol. Nevertheless, whether updates are performed once or multiple times, the chameleon hash itself does not introduce extra information leakage except the updated row i . The public row hash values published during system setup remain unchanged, and only the corresponding proof π is updated when an update occurs. By the security analysis of the chameleon hash, as proved in Section VI, update of π does not reveal the plaintext data.

VI. CORRECTNESS AND SECURITY ANALYSIS

In this section, we analyze the correctness and security of the proposed CH and ChamVerDB. We begin by demonstrating their correctness, and then formally prove collision resistance, indistinguishability, and uniqueness of CH , as well as data security and query privacy of ChamVerDB. Each security property is reduced to a standard hardness assumption (RSA or DDH), demonstrating that ChamVerDB achieves provable end-to-end security guarantees.

A. Correctness Analysis

Theorem 1: The proposed chameleon hash CH is correct.

Proof: To prove the correctness of CH , we must show that when the message m is updated to m' , $\mathcal{DO}$, who holds the corresponding chameleon hash secret key d_{ch} , can compute an updated chameleon hash proof $\pi' = (H_1(m) + \pi^{ech} - H_1(m'))^{d_{ch}} \bmod N$ such that (h, π') remains a valid pair for the new message m' under the same public key.

Actually, if π' is constructed honestly, i.e., given the original message m , the chameleon hash key pair (e_{ch}, d_{ch}) , the proof π , the chameleon hash value h , and the updated message m' , $\mathcal{DO}$ computes the new proof as $\pi' = (H_1(m) + \pi^{ech} - H_1(m'))^{d_{ch}} \bmod N$, we have $h' = g^{H_1(m')} \cdot g^{(\pi')^{ech}} \bmod N^2 = g^{H_1(m')} \cdot g^{((H_1(m) + \pi^{ech} - H_1(m'))^{d_{ch}})^{ech}} \bmod N^2 = g^{H_1(m')} \cdot g^{H_1(m) + \pi^{ech} - H_1(m')} \bmod N^2 = g^{H_1(m)} \cdot g^{\pi^{ech}} \bmod N^2 = h$. This proves the correctness of our CH scheme.

Theorem 2: The proposed ChamVerDB is correct.

Proof: To prove the correctness of ChamVerDB, we need to show that, given the query vectors q_1 and q_2 for the data at the i -th row and j -th column, $\mathcal{QU}$ can correctly retrieve the ciphertext EDB_{ij} , recover the data x_{ij} , and verify its validity.

Specifically, according to the process of ChamVerDB, to query the data at the i -th row and j -th column, $\mathcal{QU}$ generates two query vectors q_1 and q_2 , where $q_1 = (0, \dots, 0, \frac{1}{i\text{-th}}, 0, \dots, 0)_{1 \times n}^T$ and $q_2 = (0, \dots, 0, \frac{1}{j\text{-th}}, 0, \dots, 0, \frac{1}{(n+1)\text{-th}})_{1 \times (n+1)}^T$. $\mathcal{QU}$ sends q_1 to $\mathcal{CS}$ to initiate the query. Upon receiving q_1 , $\mathcal{CS}$ computes the Hadamard product with the encrypted database and returns the i -th row ciphertext $EDB_i = (EDB_{i1}, EDB_{i2}, \dots, EDB_{in}, \pi_i)$ to $\mathcal{QU}$, where $EDB_{ij} = (c_{ij} \| p_{ij})$, and c_{ij} is composed of three parts $c_{ij}^1 = (g_0)^{-1} H_2(\hat{g}^r)(1+N)^{x_{ij}} \bmod N^2$, $c_{ij}^2 = \hat{g}_t^r Z^{k_j}$, and $c_{ij}^3 = (\hat{g}^a)^{k_j}$, for $j \in [1, n]$. Then, $\mathcal{QU}$ performs a Hadamard product $q_2 \odot EDB_i$ to extract the target ciphertext $EDB_{ij} = (c_{ij}, p_{ij})$. To recover the plaintext x_{ij} , $\mathcal{QU}$ randomly selects a value α and computes the re-encryption request $c_{ij}^{3'} = c_{ij}^3 \cdot (\hat{g}^a)^\alpha = \hat{g}^{ak} \cdot \hat{g}^{a\alpha} = \hat{g}^{a(k+\alpha)}$, and sends $c_{ij}^{3'}$ to $\mathcal{CS}$.

Next, $\mathcal{CS}$ uses the re-encryption key $rk_{\mathcal{DO} \rightarrow \mathcal{QU}} = \hat{g}^{\frac{b}{a}}$ to compute the re-encrypted ciphertext $c_{ij}^{3''} = e(rk_{\mathcal{DO} \rightarrow \mathcal{QU}}, c_{ij}^{3'}) = e(\hat{g}^{\frac{b}{a}}, \hat{g}^{a(k+\alpha)}) = e(\hat{g}, \hat{g})^{b(k+\alpha)} = Z^{b(k+\alpha)}$, and sends it back to $\mathcal{QU}$.

Upon receiving $c_{ij}^{3''}$, $\mathcal{QU}$ computes $c_{ij}^{2'} = (c_{ij}^2)' \cdot Z^\alpha = \hat{g}_t^r Z^{k+\alpha}$ and reconstructs the original data x_{ij} by computing:

$$\begin{aligned} & \frac{\left(g_0 H_2 \left(\left(\frac{c_{ij}^{2'}}{(c_{ij}^{3''})^{\frac{1}{b}}} \right)^{-1} c_{ij}^1 \right) - 1 \right)}{N} \bmod N \\ &= \frac{\left(g_0 H_2 \left(\left(\frac{\hat{g}_t^r Z^{k+\alpha}}{(Z^{b(k+\alpha)})^{\frac{1}{b}}} \right)^{-1} c_{ij}^1 \right) - 1 \right)}{N} \bmod N \\ &= \frac{(g_0 H_2(\hat{g}_t^r)^{-1} c_{ij}^1) - 1}{N} \bmod N \\ &= \frac{(g_0 H_2(\hat{g}_t^r)^{-1} (g_0)^{-1} H_2(\hat{g}_t^r) (1+N)^{x_{ij}}) - 1}{N} \bmod N \\ &= \frac{(1+N)^{x_{ij}} - 1}{N} \bmod N = x_{ij}. \end{aligned}$$

Therefore, $\mathcal{QU}$ correctly recovers the plaintext x_{ij} . To further ensure validity, $\mathcal{QU}$ computes the parity of i -th row as $p_i = x_{ij} \oplus p_{ij} \oplus H(x_{ij} \| j)$, and then checks whether $h_i \stackrel{?}{=} g^{H_1(p_i)} \cdot g^{\pi_i^{ech}} \bmod N^2$ holds, using the published chameleon hash h_i . If the equality holds, the returned data is valid. Based on the above analysis, we conclude that ChamVerDB is correct.

B. Security Analysis

This section presents security proofs for both CH and ChamVerDB.

1) **Security Analysis of CH Scheme:** In this part, we first give some lemmas and proofs, then use them to prove that our CH is secure.

Lemma 1: Let $N \geq 2$ be an arbitrary integer and $e > N$ be a prime number. Then, for any $r, r' \in \mathbb{Z}_N^*$, the congruence $r^e \equiv r'^e \pmod{N}$ holds if and only if $r \equiv r' \pmod{N}$.

Proof: Since $e > N \geq \varphi(N)$ and e is prime number, it follows that $\gcd(e, \varphi(N)) = 1$. By Euler's theorem, this implies there exists an integer d such that $ed \equiv 1 \pmod{\varphi(N)}$. In particular, there exists some integer k such that $ed = k\varphi(N) + 1$. Let $r, r' \in \mathbb{Z}_N^*$ and assume that $r^e \equiv r'^e \pmod{N}$. Raising both sides to the power of d , we have $r^{ed} \equiv r'^{ed} \pmod{N}$. Substituting the expression for ed gives $r^{k\varphi(N)+1} \equiv r'^{k\varphi(N)+1} \pmod{N}$. By Euler's theorem, which states that $x^{\varphi(N)} \equiv 1 \pmod{N}$ for any $x \in \mathbb{Z}_N^*$, we conclude $r^{k\varphi(N)+1} \equiv r \pmod{N}$, $r'^{k\varphi(N)+1} \equiv r' \pmod{N}$. Thus, the congruence $r^{ed} \equiv r'^{ed} \pmod{N}$ reduces to $r \equiv r' \pmod{N}$.

The converse direction is obvious: given $r \equiv r' \pmod{N}$, we have $r^e \equiv r'^e \pmod{N}$. This completes the proof.

Lemma 2 (Collision-Resistance of CH): Under Definition 1, the proposed CH ensures Collision-Resistance.

Proof: To prove Collision-Resistance of CH, we define several hybrid games:

- **Game₀:** This game follows the standard execution of CH, denoted by event E_{Game_0} .
- **Game₁:** This game is identical to **Game₀** except that an adversary $\mathcal{A}$ attempts to find a collision. Specifically, given a message m and its corresponding chameleon hash pair (h, π) , the adversary $\mathcal{A}$ aims to find a different message m' and a new randomness π' such that the verification equation still holds: $\text{CH.Check}(pk, m, \pi, h) = \text{CH.Check}(pk, m', \pi', h)$. We denote this event by E_{Game_1} .

Transition. Game₀ $\rightarrow$ Game₁: Assume there exists an algorithm $\mathcal{B}$ capable of breaking the standard RSA assumption. Given a message m and the corresponding CH output (h, π) satisfying $h = g^{H(m)} g^{\pi^{e_{ch}}} \pmod{N^2}$, the adversary $\mathcal{A}$, using $\mathcal{B}$, attempts to find another message m' and randomness π' such that $h = g^{H(m')} g^{\pi'^{e_{ch}}} \pmod{N^2}$, i.e., $\mathcal{A}$ finds a collision. Equating the two expressions for h yields $g^{H(m)} g^{\pi^{e_{ch}}} \equiv g^{H(m')} g^{\pi'^{e_{ch}}} \pmod{N^2}$, which implies that $\pi^{e_{ch}} \equiv H(m) - H(m') + \pi'^{e_{ch}} \pmod{N}$. Letting $y = H(m) - H(m') + \pi'^{e_{ch}} \pmod{N}$, it follows that $\pi^{e_{ch}} \equiv y \pmod{N}$. By Euler's theorem, the unique solution is given by $\pi' = y^{d_{ch}} \pmod{N}$, which indicates that π' can be computed only if the secret exponent d_{ch} is known. Therefore, if the adversary $\mathcal{A}$ can successfully find (m', π') satisfying the above condition, it effectively enables $\mathcal{B}$ to compute d_{ch} , thereby breaking the RSA assumption. Consequently, we have $|\Pr[E_{\text{Game}_1}] - \Pr[E_{\text{Game}_0}]| \leq \text{Adv}_{\mathcal{A}}^{\text{RSA}}(\mathcal{B})$, where $\text{Adv}_{\mathcal{A}}^{\text{RSA}}(\mathcal{B})$ denotes $\mathcal{A}$'s advantage in finding a chameleon hash collision through $\mathcal{B}$'s ability to break RSA assumption.

Hence, under the standard RSA assumption, no PPT adversary can generate a chameleon hash collision without knowledge of the secret key. The above analysis means that our CH scheme satisfies Collision-Resistance.

Lemma 3 (Indistinguishability of CH): Under Definition 2, the proposed CH ensures Indistinguishability.

Proof: To prove the indistinguishability of the proposed CH, we define the following hybrid games:

- **Game₀:** This game corresponds to the original execution of CH. Given a message m and public key pk_{ch} , the challenger computes $(h, \pi) \leftarrow \text{CH.Hash}(pk_{ch}, m)$ and returns (h, π) . Let E_{Game_0} denote the event that the adversary knows this output.
- **Game₁:** This game proceeds similarly to **Game₀**, except that after computing (h, π) as before, the challenger samples a fresh message $m' \neq m$ and computes $\pi' \leftarrow \text{CH.Adapt}(sk, m, m', \pi, h)$ using the secret key sk . The resulting output is then (h, π') . Let E_{Game_1} denote the event that the adversary distinguishes between the distributions of π and π' .

Transition. Game₀ $\rightarrow$ Game₁: Note that in CH.Hash , the randomness π is sampled uniformly at random. In contrast, in CH.Adapt , the randomness π' is deterministically derived from (m, m', π, h) using the trapdoor sk . However, based on $\pi' = (H_1(m) + \pi^{e_{ch}} - H_1(m'))^{d_{ch}} \pmod{N}$, π and π' are computationally indistinguishable to any PPT adversary who does not know sk . Therefore, we have $|\Pr[E_{\text{Game}_1}] - \Pr[E_{\text{Game}_0}]| \leq \text{negl}(\lambda)$, where $\text{negl}(\lambda)$ denotes a negligible function in the security parameter λ .

The above analysis completes the proof of the Indistinguishability of CH.

Lemma 4 (Uniqueness of CH): Under Definition 3, the proposed CH ensures uniqueness.

Proof: To prove Uniqueness of CH, we define the following hybrid games:

- **Game₀:** This game corresponds exactly to the original execution process of the CH scheme, and we denote this event as E_{Game_0} .
- **Game₁:** This game is the same as **Game₀**, except that the challenger aborts if, for a given public key pk_{ch} , message m , randomness π , and the corresponding hash value h , the adversary $\mathcal{A}$ can find another randomness $\pi' \neq \pi$ such that the chameleon hash check still holds, i.e., $\text{CH.Check}(pk_{ch}, m, \pi', h) = \text{CH.Check}(pk_{ch}, m, \pi, h)$. This event is denoted as E_{Game_1} .

Transition. Game₀ $\rightarrow$ Game₁: Suppose, for contradiction, that the adversary $\mathcal{A}$ is able to find a distinct randomness π' such that $h = g^{H(m)} \cdot g^{\pi'^{e_{ch}}} \pmod{N^2}$. By the construction of the CH scheme, we also have $h = g^{H(m)} \cdot g^{\pi^{e_{ch}}} \pmod{N^2}$. Equating the two expressions yields $g^{\pi'^{e_{ch}}} \equiv g^{\pi^{e_{ch}}} \pmod{N^2}$, which implies that $\pi'^{e_{ch}} \equiv \pi^{e_{ch}} \pmod{N}$. By Lemma 1, this congruence holds if and only if $\pi \equiv \pi' \pmod{N}$. However, this contradicts the assumption that π' is a fresh and distinct randomness. Therefore, the transition from **Game₀** to **Game₁** occurs only with negligible probability. Consequently, we have $|\Pr[E_{\text{Game}_1}] - \Pr[E_{\text{Game}_0}]| \leq \text{negl}(\lambda)$.

Hence, the CH scheme satisfies the Uniqueness property as defined in Definition 3.

Theorem 3: Our CH scheme is secure under the random oracle model.

Proof: The properties of Collision-Resistance, Indistinguishability, and Uniqueness are established in Lemmas 2, 3, and 4, respectively. Therefore, we conclude that our CH scheme is secure.

2) *Security Analysis of ChamVerDB*: In this part, we prove that our ChamVerDB ensures data security and guarantees query privacy.

Theorem 4 (Data security of ChamVerDB): Let $\mathcal{A}$ be an adversary against ChamVerDB. If the DDH problem (Definition 4) is hard, ChamVerDB guarantees data security.

Proof: This proof proceeds by gradually replacing the correlated components with random group elements and showing that each replacement is computationally indistinguishable under the DDH assumption. In the final hybrid, the challenge ciphertext is independent of the challenge bit, and hence the adversary obtains no advantage. Specifically, assume that there exists a $\mathcal{PPT}$ adversary $\mathcal{A}$ capable of compromising the security of our ChamVerDB scheme with non-negligible probability. Then, we can construct a challenger $\mathcal{C}$ that leverages this advantage to solve the DDH assumption (Definition 4). Specifically, given a DDH instance, $\mathcal{C}$ aims to determine whether the element is g^{ab} or a random element sampled uniformly from $\mathbb{G}_T$ (or a random element in $\mathbb{G}$). This construction demonstrates that if $\mathcal{A}$ can successfully break the security of ChamVerDB, the corresponding $\mathcal{C}$ can utilize this capability to solve the DDH assumption.

- H_0 : This hybrid represents the original security game.
- H_1 : This hybrid is identical to H_0 except in the generation of the challenge ciphertexts. Specifically, the challenger generates the ciphertext ct_b^* as follows:
 - Sample $b \xleftarrow{\$} \{0, 1\}$ and $k \xleftarrow{\$} \mathbb{Z}_N^*$.
 - For index b , compute $ct_b^{1*} = g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}$, $ct_b^{2*} = \hat{g}_t^r Z^k$, and $ct_b^{3*} = \hat{g}^{ak}$, as in H_0 .
 - For index $1-b$, choose $k' \xleftarrow{\$} \mathbb{Z}_N^*$ and $h_g \in \mathbb{G}$ uniformly at random, and compute $ct_{1-b}^{1*} = g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}$, $ct_{1-b}^{2*} = \hat{g}_t^r Z^{k'}$, and $ct_{1-b}^{3*} = h_g$.
 - Set $ct_b^* = (ct_b^{1*}, ct_b^{2*}, ct_b^{3*})$.

In H_0 and H_1 , the only difference is the replacement of the second and third ciphertext components by random elements. From the $\mathcal{A}$'s perspective, the ciphertext $(g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}, \hat{g}_t^r Z^{k'}, h_g)$ in H_1 is computationally indistinguishable from $(g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}, \hat{g}_t^r Z^k, \hat{g}^{ak})$ in H_0 under DDH assumption. Therefore, hybrids H_0 and H_1 are computationally indistinguishable from $\mathcal{A}$'s point of view.

- H_2 : This hybrid is identical to H_1 except for the method used to generate the challenge ciphertexts. In this hybrid, the challenger generates ct_b^* and ct_{1-b}^* as follows:
 - Sample $b \xleftarrow{\$} \{0, 1\}$ and randomly choose $h'_g, h_g \xleftarrow{\$} \mathbb{G}$.
 - For index b , choose $k' \xleftarrow{\$} \mathbb{Z}_N^*$ and compute $ct_b^{1*} = g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}$, $ct_b^{2*} = \hat{g}_t^r Z^{k'}$, and $ct_b^{3*} = h_g$.
 - For index $1-b$, select a random element $h_Z \in \mathbb{G}_T$, set $ct_{1-b}^{2*} = h_Z$ and $ct_{1-b}^{3*} = h'_g$, compute $ct_{1-b}^{1*} = g_0 H_2(h_Z)(1+N)^{x_b}$.
 - Set $ct_b^* = (ct_b^{1*}, ct_b^{2*}, ct_b^{3*})$.

In H_0 and H_1 , it is straightforward to see that the challenge ciphertext ct_b^* is modified in the same manner as ct_{1-b}^* in hybrid H_1 . Specifically, ct_b^{2*} and ct_b^{3*} are changed in H_2 , while all three components of ct_{1-b}^* are modified: ct_{1-b}^{2*} and ct_{1-b}^{3*} are chosen uniformly at random, and ct_{1-b}^{1*} is

computed accordingly. From $\mathcal{A}$'s perspective, the ciphertext $(g_0 H_2(\hat{g}_t^r)(1+N)^{x_b}, \hat{g}_t^r Z^{k'}, h_g)$ in H_1 is computationally indistinguishable from $(g_0 H_2(h_Z)(1+N)^{x_b}, h_Z, h'_g)$ in H_2 under the DDH assumption. Hence, we conclude that H_1 and H_2 are computationally indistinguishable to $\mathcal{A}$. Finally, combining the above hybrid transitions, we conclude that ChamVerDB guarantees data security under the DDH assumption. This completes the proof.

Definition 5 (Formal definition of Query privacy): Let $\Pi_{\text{ChamVerDB}} = (\text{Setup}, \text{Preprocess}, \text{Outsource}, \text{Query}, \text{Answer}, \text{Recon}, \text{Verify})$ be a verifiable database scheme. Given a security parameter λ , database $DB = \{x_m\}_{[1, n^2]} = (x_{ij})_{n \times n}$, where $m = (i-1) \times n + j$ for $i, j \in [1, n]$. ChamVerDB reveals the queried row index through the row-retrieval mechanism while hiding the column index. Accordingly, the privacy notion here is for any fixed row index i , the adversary should not be able to distinguish queries to different columns in that row beyond negligible advantage.

Specifically, given an adversary $\mathcal{A}$, we define the distributions of the real execution of the protocol and the ideal execution for a simulator $\mathcal{S}$ as

$$\text{REAL}_{\mathcal{A}, DB, m, \lambda} := \left\{ x_m : \begin{array}{l} (PP, EDB, rk) \leftarrow \text{Init}(1^\lambda, DB) \\ (q_1, q_2) \leftarrow \text{Query}(m = (i, j)) \\ ct^* \leftarrow \mathcal{A}(DB, q_1) \\ req \leftarrow \text{ReqREC}(ct_{ij}^*, PP) \\ rec \leftarrow \mathcal{A}(rk, req) \\ x_m \leftarrow \text{Dec}(rec, ct_{ij}^*) \end{array} \right\},$$

and

$$\text{IDEAL}_{\mathcal{A}, \mathcal{S}, DB, \lambda} := \left\{ x'_m : \begin{array}{l} (PP, EDB, rk) \leftarrow \text{Init}(1^\lambda, DB) \\ (q_1, q'_2) \leftarrow \mathcal{S.QRY}(PP, q_1) \\ ct^* \leftarrow \mathcal{A}(DB, q_1) \\ req' \leftarrow \mathcal{S.REQ}(PP) \\ rec' \leftarrow \mathcal{A}(rk, req') \\ x'_m \leftarrow \text{Dec}(rec', ct_{ij}^*) \end{array} \right\},$$

respectively, where the offline phase is represented as Init, and the Recon phase is divided into two subphases: ReqREC and Dec. ChamVerDB guarantees query privacy if, for any adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that for any given database DB , $\mathcal{A}$ cannot distinguish, with non-negligible probability, whether the data is produced by the real-world execution $\text{REAL}_{\mathcal{A}, DB, m, \lambda}$ or by the ideal-world simulation $\text{IDEAL}_{\mathcal{A}, \mathcal{S}, DB, \lambda}$, i.e.,

$$|\Pr[\text{REAL}_{\mathcal{A}, DB, m, \lambda} = 1] - \Pr[\text{IDEAL}_{\mathcal{A}, \mathcal{S}, DB, \lambda} = 1]| \leq \text{negl}(\lambda).$$

Theorem 5 (Query privacy of ChamVerDB): Our proposed ChamVerDB guarantees query privacy.

Proof: Given a database DB and the index $m = (i, j)$ defined in Definition 5, for any adversary $\mathcal{A}$, we define the re-encryption request procedure as **Procedure** and the **Simulator** $\mathcal{S}$, where $\mathcal{S.QRY}(PP)$ and $\mathcal{S.REQ}(PP)$ represent the simulator's actions in the query and re-encryption request steps, respectively. In both steps, $\mathcal{S}$ outputs random data instead of executing the corresponding operations in ChamVerDB.

We first give an analysis sketch. In ChamVerDB, the adversary $\mathcal{A}$ observes the public parameters, the outsourced encrypted

database, the re-encryption keys, the row query vector q_1 , and the re-encryption request sent by $\mathcal{QU}$. However, the local selector q_2 is never sent to $\mathcal{A}$. Therefore, once the row index is fixed, it suffices to show that the adversary's view is independent of the hidden queried column. Meanwhile, $\mathcal{S}$ preserves the intentionally revealed row query vector q_1 and replaces only the hidden client-side components. Concretely, it first replaces the local query vector q_2 with a random vector q'_2 , and then replaces the re-encryption request with a random group element. We show that each replacement does not change $\mathcal{A}$'s view beyond negligible advantage.

Procedure $ReqREC(ct_{ij}^{3*}, PP)$

- 1: randomly select $\alpha \xleftarrow{\$} \mathbb{Z}_N^*$
 - 2: compute $ct_{ij}^{3*'} = ct_{ij}^{3*} \cdot (\hat{g})^\alpha$
 - 3: set $ct_{ij}^{3*'}$ as req
 - 4: return req
-

Simulator $\mathcal{S.QRY}(PP, q_1)$

- 1: Initialize $q'_2 \in \{0\}^{n+1}$.
 - 2: $j' \xleftarrow{\$} [1, n]$
 - 3: set $q'_2[j'] = 1$, $q'_2[n+1] = 1$
 - 4: **return** (q_1, q'_2)
-

Simulator $\mathcal{S.REQ}(PP)$

- 1: randomly select $\gamma \xleftarrow{\$} \mathbb{G}$
 - 2: set γ as req'
 - 3: **return** req'
-

In detail, to prove that the real distribution $\text{REAL}_{\mathcal{A}, DB, m, \lambda}$ and the ideal distribution $\text{IDEAL}_{\mathcal{A}, S, DB, \lambda}$ are computationally indistinguishable, we define a sequence of hybrid experiments:

- H_0 : This hybrid is the real execution $\text{REAL}_{\mathcal{A}, DB, m, \lambda}$.
 - Initialize the system, generate key pairs and re-encryption keys, publish PP , hash values, and EDB .
 - The challenger constructs q_1 and q_2 according to $m = (i, j)$ and sends q_1 to $\mathcal{A}$.
 - $\mathcal{A}$ responds with ct^* .
 - The challenger parses ct^* as $(ct_{i1}^*, \dots, ct_{in}^*, \pi_i)$, computes $q_2 \odot ct^*$ to obtain (ct_{ij}^*, π_i) , parses ct_{ij}^* as $(ct_{ij}^{1*}, ct_{ij}^{2*}, ct_{ij}^{3*})$, and executes $req \leftarrow ReqREC(ct_{ij}^{3*}, PP)$.
 - $\mathcal{A}$ returns $ct_{ij}^{3*'}$ and the challenger reconstructs x_m as in ChamVerDB.
- H_1 : Identical to H_0 except that the challenger keeps q_1 unchanged and replaces q_2 with $(q_1, q'_2) \leftarrow \mathcal{S.QRY}(PP, q_1)$, where q'_2 corresponds to a uniformly random column index $j' \in [1, n]$.
- H_2 : Identical to H_1 except that the challenger computes $req' \leftarrow \mathcal{S.REQ}(PP)$ instead of executing $ReqREC$.

Obviously, in H_1 and from $\mathcal{A}$'s perspective, the local vector q_2 is never revealed to it. Even if we relax the constraint on

q_1 , $\mathcal{A}$ still cannot distinguish whether the index in q_1 is a real intention or a random choice. Thus, we can infer that H_0 and H_1 are computationally indistinguishable. Furthermore, In H_2 , the re-encryption request is $req = ct_{ij'}^{3*} \cdot \hat{g}^\alpha$, where $\alpha \xleftarrow{\$} \mathbb{Z}_N^*$. Once the returned row is fixed, $ct_{ij'}^{3*}$ is a fixed group element. Multiplication by the fresh random mask $\hat{g}^\alpha$ therefore re-randomizes the request in the group. As a result, from the viewpoint of $\mathcal{A}$, req is computationally indistinguishable from a uniformly random element $\gamma \xleftarrow{\$} \mathbb{G}$ output by $\mathcal{S.REQ}(PP)$. This implies that H_1 and H_2 are also indistinguishable. Therefore, we conclude that ChamVerDB ensures query privacy.

Since H_2 coincides with the ideal execution, $\mathcal{A}$ learns no information about the hidden queried column beyond the revealed row index. Therefore, we can conclude that ChamVerDB guarantees query privacy.

VII. PERFORMANCE EVALUATION

This section evaluates the communication overhead and computational cost of our ChamVerDB and compares it with the most relevant works, i.e., PPVDB [13], APIR [31], and VUSDB [18].

A. Experiment Setting

1) *Comparison Schemes*: To evaluate the effectiveness of ChamVerDB, we compare it with three representative baselines: PPVDB [13], VUSDB [18] and APIR [31]. All schemes follow a common system model. Except for APIR [31], which operates on a plaintext database, all others are designed for ciphertext-based database retrieval. For PPVDB [13], $\mathcal{DO}$ encrypts the database, computes a commitment, and outsources the encrypted database to $\mathcal{CS}$, completing the offline phase. During the online phase, $\mathcal{QU}$ issues an obfuscated query and receives a ciphertext together with a proof as the response, then verifies it. VUSDB [18] follows essentially the same workflow as PPVDB [13], except that VUSDB [18] transmits the exact index during the online query. In APIR [31], $\mathcal{DO}$ and $\mathcal{CS}$ are the same roles, holding a plaintext database. Specifically, $\mathcal{CS}$ computes a digest over the plaintext database during the offline phase, and the online phase aligns with that of PPVDB [13] and VUSDB [18]. In the implementations, the group orders of $\mathbb{G}$ and $\mathbb{G}_T$ are set to 512 bits, $\mathbb{Z}_N$ to 1024 bits, $\mathbb{Z}_P$ (both in PPVDB [13] and VUSDB [18]) to 512 bits. In ChamVerDB, the outputs of the hash functions H_1 and H_2 are 256 bits and 2048 bits, respectively, and l is set to $\log_2 n$, where n denotes the database size.

2) *Datasets*: To ensure consistency in performance evaluation, we uniformly represent the database size as n , although the original description of ChamVerDB defines it as n^2 . To prove that our scheme achieves efficiency, reliability, and practical utility, we evaluated all the schemes on the real-world network security dataset, `ipsum`¹, which aggregates multiple public threat intelligence feeds. We extract 2^8 to 2^{20} IPv4 addresses to form different database. For easy to compute, we transform each IPv4 address from its dot-decimal string format (e.g., "1.2.3.4")

¹<https://github.com/stamparm/ipsum>

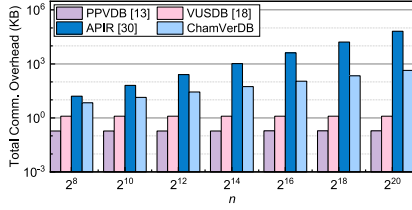


Fig. 4. Communication overhead in the online phase.

into its native 32-bit unsigned integer representation. This approach ensures a lossless mapping of the network metadata, thereby directly matching the 32-bit record requirement of our experimental settings and precisely simulating high-frequency security metadata retrieval scenarios such as privacy-preserving blacklisting.

3) *Environment*: All experiments are conducted on a server running Ubuntu 20.04.6, equipped with an IntelXeon Gold CPU@ 2.10 GHz and 192 GB of RAM. The implementations of all schemes are carried out in C++ using the Pairing-Based Cryptography (PBC) library.² The source code of the experiments is available on GitHub.³

B. Communication Overhead

APIR [31] outsources the database in plaintext, whereas ChamVerDB, PPVDB [13], and VUSDB [18] encrypt the databases before or during outsourcing. Since the offline database outsourcing phase is executed only once and can be performed offline, our analysis focuses only on the communication overhead of the frequently executed online query phase.

To ensure data privacy and result verifiability, ChamVerDB and PPVDB [13] require four communication rounds, whereas APIR [31] and VUSDB [18] complete verifiable queries in two rounds. Specifically, in ChamVerDB, QU sends a $\sqrt{n}$ -bit one-hot query vector q_1 to CS , which returns the corresponding row EDB_i consisting of $\sqrt{n}$ ciphertexts and a proof. Each ciphertext contains $|\mathbb{Z}_{N^2}^*| + |\mathbb{G}| + |\mathbb{G}_T|$ bits, together with a 32-bit parity tag. Subsequently, QU issues a re-encryption request of size $|\mathbb{G}|$ and receives a ciphertext of size $|\mathbb{G}_T|$. In PPVDB [13], QU sends an obfuscated index $v_i = i + r$, and CS returns a ciphertext EDB_{v_i} together with a proof π of size $|\mathbb{G}|$. Then, QU issues a decryption request of size $|\mathbb{Z}_p|$ and receives a ciphertext of the same size. In APIR [31], QU sends a query vector of size $n|\mathbb{G}|$ and receives a response of size $|\mathbb{G}|$. In VUSDB [18], QU sends a query tuple of size $5|\mathbb{G}|$ along with an index, and receives a tuple of size $2|\mathbb{Z}_p| + 7|\mathbb{G}|$.

Based on the above analysis, Fig. 4 presents the communication overhead of the online phase. As shown, the overhead scales differently with the dataset size n . At $n = 2^{20}$, PPVDB and VUSDB show the best communication performance, and the overhead is 0.189 KB and 1.25 KB, respectively, and almost remains unchanged with n grows. ChamVerDB yields an acceptable communication of 432 KB. However, APIR generates

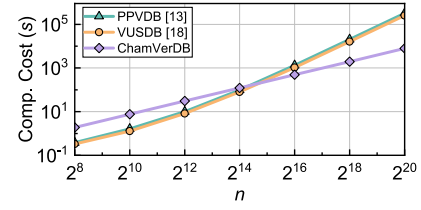


Fig. 5. Computational cost in the offline phase.

up to 65536 KB of overhead for the same amount of data. These differences reflect the underlying design trade-offs: PPVDB [13] minimizes communication by returning only a single ciphertext and proof through index randomization and interpolation. VUSDB [18] maintains low bandwidth while does not ensure query privacy. ChamVerDB preserves query privacy at the cost of returning $\sqrt{n}$ ciphertexts, whereas APIR [31] suffers from high overhead because its query vector grows linearly with n .

C. Computational Costs

We evaluate the computational efficiency of ChamVerDB in comparison with PPVDB [13], APIR [31], and VUSDB [18] in both the offline and online phases. Since APIR [31] stores the database in plaintext and does not involve preprocessing, the offline-phase comparison is conducted only against PPVDB [13] and VUSDB [18]. In the offline phase, ChamVerDB incurs only $O(\sqrt{n})$ modular operations, together with lightweight hash and XOR computations, thus avoiding the $O(n)$ -scale interpolation and encryption overhead required by PPVDB [13] and VUSDB [18]. In the online phase, ChamVerDB requires only constant or $O(\sqrt{n})$ operations, while the related schemes rely on costly modular exponentiations and pairings over n data.

1) *Offline Computational Costs*: Fig. 5 presents the preprocessing time of different schemes. Although the preprocessing time of all schemes increases with n , ChamVerDB exhibits a significantly lower growth rate. For relatively small databases ($n < 2^{15}$), PPVDB [13] and VUSDB [18] are faster. For example, at $n = 2^8$, ChamVerDB requires 1.89 s, whereas VUSDB [18] and PPVDB [13] take 0.16 s and 0.03 s, respectively. However, this advantage diminishes as the database size grows. At $n = 2^{20}$, PPVDB [13] and VUSDB [18] require 331234 s and 254018 s, respectively, while ChamVerDB completes preprocessing in only 7856.64 s, achieving approximately 42 $\times$ and 32 $\times$ speedups over PPVDB [13] and VUSDB [18], respectively. This improvement mainly stems from the different computational strategies adopted by the three schemes. ChamVerDB encrypts each record into three ciphertext components and generates one chameleon hash for each row, resulting in substantially lower preprocessing overhead. By contrast, PPVDB [13] relies on Lagrange polynomial interpolation over all n data points, which incurs $O(n^2)$ complexity. VUSDB [18] requires both the $\mathcal{DO}$ and the $\mathcal{CS}$ to perform $O(n)$ operations. Therefore, PPVDB [13] and VUSDB [18] incur substantially higher preprocessing overhead at large database scales.

2) *Online Computational Costs*: Fig. 6 shows the online performance of different schemes. For clarity, we combine the

²<https://crypto.stanford.edu/pbc>

³<https://github.com/SkywalkerQi/ChamVerDB>

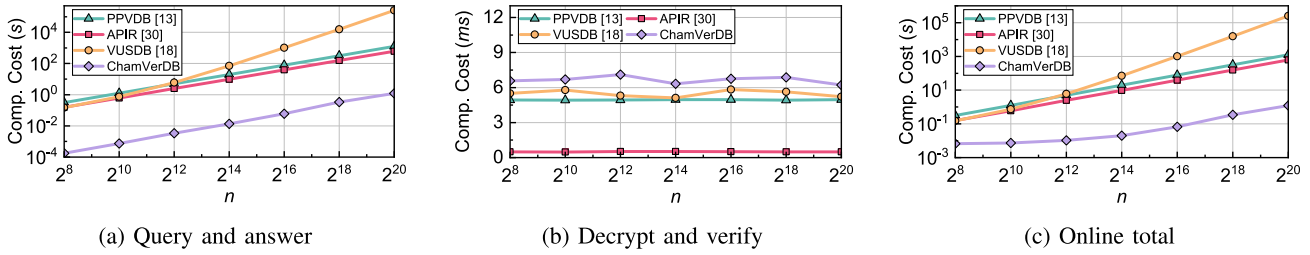


Fig. 6. Comparison of computational cost for different schemes in online phase.

Query and Answer steps, as well as the Reconstruct and Verify steps, in the comparison. As shown in Fig. 6(a), the cost of query and answer increases with the database size n for all schemes. Among them, VUSDB [18] incurs the highest cost, followed by PPVDB [13] and APIR [31], whereas ChamVerDB remains highly efficient. For example, at $n = 2^{20}$, VUSDB [18], PPVDB [13], and APIR [31] require 253871.5 s, 1258.14 s, and 618.5 s, respectively, while ChamVerDB completes the same process in only 1.2 s. This difference mainly arises from the distinct query-response mechanisms of the schemes. In VUSDB [18], although the ciphertext can be directly obtained, $\mathcal{CS}$ must perform an expensive proof computation over the entire database. In PPVDB [13], the query itself is lightweight, but the response generation requires Lagrange interpolation over n data points, leading to substantial overhead. In APIR [31], $\mathcal{QU}$ constructs an n -dimensional query vector and performs modular multiplications and exponentiations on each dimension, while $\mathcal{CS}$ carries out additional modular multiplications to answer the query. By contrast, in ChamVerDB, $\mathcal{QU}$ only constructs two binary vectors of lengths n and $n + 1$, and $\mathcal{CS}$ responds by a Hadamard product, which is equivalent to selecting the target row from the encrypted database. Consequently, ChamVerDB achieves highly efficient query and answer processing.

Fig. 6(b) compares the computational costs of the Reconstruct and Verify steps among the four schemes. These costs remain nearly constant as the database size n increases. APIR [31] is the most efficient, since it involves only a few modular multiplications and inversions. PPVDB [13] requires three bilinear pairings, two modular multiplications, and two modular exponentiations, while VUSDB [18] involves ten bilinear pairings, three modular multiplications, one modular exponentiation, one modular addition, and one hash operation. In ChamVerDB, reconstruction requires three modular exponentiations, five modular multiplications, one bilinear pairing, one hash operation, three modular inverses, and one modular addition, whereas verification involves three modular exponentiations, one modular multiplication, two hash operations, and two XOR operations. Although ChamVerDB incurs a slightly higher cost than other schemes, the total cost remains at the microsecond level and is therefore negligible in practice.

Finally, Fig. 6(c) summarizes the total online computational cost of the four schemes. Since the Reconstruct and Verify steps incur only negligible overhead, the overall trend is nearly identical to that of the Query and Answer steps and we omit the repetitive description. ChamVerDB consistently achieves

the best online performance, and its advantage becomes more pronounced as the database size n increases. This result further confirms the superior scalability and efficiency of ChamVerDB in large-scale settings.

D. Other Evaluations

1) *Evaluation Under Different Network Settings:* To further evaluate the practical effect of different online communication payloads, we measured the end-to-end query latency under several network settings. In addition to the default LAN setting, we used the Linux `tc` command to emulate bandwidth-constrained links of 100 Mbps, 10 Mbps, and 1 Mbps between $\mathcal{QU}$ and $\mathcal{CS}$. We set the round-trip time (RTT) to 1 ms for LAN, and to 80 ms for WAN settings. Table I shows the end-to-end online latency of all schemes under different network settings. The results show that the practical performance of each scheme is jointly determined by its computational and communication costs. Under LAN settings, where communication delay is negligible, ChamVerDB consistently achieves the best online performance throughout the evaluation. These results indicate that, although ChamVerDB needs to transmit an entire encrypted row in the online phase, its total running time remains well controlled in practice, and its much lower computational cost continues to dominate the end-to-end latency. APIR [31] is inferior to ChamVerDB, as it suffers from a heavy communication overhead resulting from the transfer of n data. By contrast, the heavy cryptographic computation of PPVDB [13] and VUSDB [18] lead to significantly higher total latency. As the database size grows, our advantage becomes even more pronounced. For example, when $n = 2^{20}$, ChamVerDB only requires 4.75 s under 1 Mbps settings, whereas APIR [31] requires 1155.40 s, PPVDB [13] and VUSDB [18] remain prohibitively expensive. These results clearly illustrate the trade-off between computation and communication in our design: although ChamVerDB incurs higher online communication than schemes with very small response sizes, its much lower computational cost makes it the most practical choice for large-scale deployments, even under constrained network conditions.

2) *Evaluation Under Different Payloads:* To evaluate the impact of payload size, we fix the database size to $n = 2^{12}$, a bandwidth of 100 Mbps with an RTT of 80 ms, and vary the payload length from 8 bits to 128 bits. Table II illustrates the corresponding online running times of all schemes.

TABLE I
COMPARISON OF COMM. AND COMP. COSTS UNDER DIFFERENT BANDWIDTH SETTINGS. THE BEST RESULTS ARE MARKED IN RED

Scheme	n	Online Running Time (s)				n	Online Running Time (s)			
		LAN	100Mbps	10Mbps	1Mbps		LAN	100Mbps	10Mbps	1Mbps
PPVDB [13]	2^{10}	1.24	1.32	1.32	1.22	2^{12}	4.94	5.02	5.02	5.02
APIR [31]		0.60	0.68	0.68	1.12		2.47	2.55	2.68	4.57
VUSDB [18]		0.74	0.83	0.83	0.83		6.05	6.13	6.13	6.13
Ours		0.01	0.09	0.09	0.12		0.01	0.09	0.09	0.23
PPVDB [13]	2^{14}	19.56	19.64	19.64	19.64	2^{16}	78.20	78.28	78.28	78.28
APIR [31]		9.67	9.76	10.51	18.06		39.18	39.52	42.54	72.74
VUSDB [18]		70.47	70.55	70.55	70.55		1010.71	1010.79	1010.79	1010.79
Ours		0.02	0.10	0.10	0.46		0.07	0.15	0.16	0.95
PPVDB [13]	2^{18}	313.77	313.84	313.84	313.84	2^{20}	1258.14	1258.22	1258.22	1258.22
APIR [31]		157.11	158.46	170.54	291.34		618.52	623.90	672.21	1155.40
VUSDB [18]		15752	15752	15752	15752		253871	253871	253871	253871
Ours		0.34	0.43	0.52	2.12		1.22	1.29	1.57	4.75

TABLE II
COMP. COSTS UNDER DIFFERENT PAYLOADS WITH $n = 2^{12}$

Scheme	8bit	16bit	32bit	64bit	128bit
APIR [31]	5.0518	5.0148	5.0173	5.0530	5.0143
PPVDB [13]	2.1588	2.2424	2.4997	3.0115	3.9157
VUSDB [18]	6.1142	6.1274	6.1361	6.1419	6.5602
Ours	0.0940	0.0947	0.0941	0.0947	0.0946

The results show that ChamVerDB consistently achieves the lowest running time across all payload sizes, and its performance remains highly stable as the payload grows. Specifically, its online latency is maintained at 0.094 s when the payload increases from 8 bits to 128 bits. APIR [31] is also relatively stable, but remains much slower than ChamVerDB. In contrast, PPVDB [13] and VUSDB [18] exhibit more noticeable performance degradation as the payload increases, especially PPVDB [13]. The larger increase of PPVDB [13] mainly comes from its interpolation-based response generation. Since the server must perform Lagrange interpolation over n data points, larger payloads lead to heavier arithmetic values and thus noticeably higher running time. VUSDB [18] is also affected by payload growth, but less significantly, because its dominant overhead already lies in expensive proof computation over the database. While, ChamVerDB is dominated by row retrieval and proxy re-encryption, making it far less sensitive to payload size.

E. Summary of Performance

To ensure security and privacy, higher costs of communication and computation are often inevitable, particularly with large-scale datasets, making a secure and efficient verifiable outsourced database especially challenging. A practical VDB must strike a balance between communication and computation, as well as between security and efficiency. Our performance analysis demonstrates that when the real-world database size

reaches 2^{20} with 100 Mbps bandwidth and RTT of 80 ms, although VUSDB [18] achieves an extremely low communication cost of only 1.25 KB, it demands 253871 s to complete a single query, making it practically infeasible for real-world applications. PPVDB [13] incurs only 0.19 KB in communication but requires 1258.22 s to complete a query. APIR [31], with 65536 KB communication and 623.90 s of computation time, also suffers from significant overhead. In contrast, ChamVerDB achieves an optimal trade-off, requiring just 432 KB of communication and 3.07 s of computation. Notably, while competing baselines encounter scalability limits as data volume increases, only ChamVerDB maintains the efficiency to successfully execute on massive datasets up to $n = 2^{26}$. By reducing query latency from hours or tens of minutes to sub-second levels with only hundreds of KB of communication, ChamVerDB greatly enhances user experience and demonstrates strong practicality and superiority in verifiable database applications. It is thus particularly well-suited for verifiable database query scenarios demanding both efficiency and security.

VIII. CONCLUSION

This paper proposes ChamVerDB, a chameleon hash-based verifiable outsourced database scheme. By combining modified BG encryption with proxy re-encryption, ChamVerDB enables secure queries while preserving data privacy. Its matrix-based structure supports localized ciphertext operations, reducing computational cost, and the chameleon hash allows lightweight result verification. Security analysis and performance evaluation show that ChamVerDB ensures data security, query privacy, verifiability, and efficiency. Despite its promising performance, challenges remain. ChamVerDB incurs considerable communication overhead, which could be mitigated by returning fewer than $\sqrt{n}$ ciphertexts or applying compression techniques. Besides, supporting multi-table queries via JOINS or subqueries is difficult due to encrypted cross-table operations, complex conditions, potential leakage, and higher overhead. Developing an efficient, verifiable framework for such queries will be our future work.

REFERENCES

- [1] S. Benabbas, R. Gennaro, and Y. Vahlis, "Verifiable delegation of computation over large datasets," in *Proc. 31st Annu. Cryptol. Conf.*, Santa Barbara, CA, USA, 2011, pp. 111–131.
- [2] M. Miao, J. Wang, J. Ma, and W. Susilo, "Publicly verifiable databases with efficient insertion/deletion operations," *J. Comput. System Sci.*, vol. 86, pp. 49–58, 2017.
- [3] W. Zhou, Y. Cai, Y. Peng, S. Wang, K. Ma, and F. Li, "VeriDB: An SGX-based verifiable database," in *Proc. Int. Conf. Manage. Data*, New York, NY, USA: Association for Computing Machinery, 2021, pp. 2182–2194.
- [4] H. Xia and M. Wang, "LightPIR: Single-server PIR via FHE without gaussian noise," in *Proc. 19th ACM Asia Conf. Comput. Commun. Secur.*, 2024, pp. 1049–1065.
- [5] A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo, "Batch PIR and labeled PSI with oblivious ciphertext compression," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 5949–5966.
- [6] I. Demertzis, J. G. Chamani, D. Papadopoulos, and C. Papamanthou, "Dynamic searchable encryption with small client storage," in *Proc. 27th Annu. Netw. Distrib. System Secur. Symp.*, 2020, pp. 1–18.
- [7] S.-F. Sun et al., "Practical non-interactive searchable encryption with forward and backward privacy," in *Proc. Usenix Netw. Distrib. System Secur. Symp.*, 2021, pp. 1–18.
- [8] P. Xu et al., "ROSE: Robust searchable encryption with forward and backward security," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1115–1130, 2022.
- [9] W. Q. Zhang, X. Li, Z. M. Yin, W. Liang, K. Huang, and X. S. Zhang, "Robust scheme for conjunctive symmetric searchable encryption with forward and backward privacy," *Ruan Jian Xue Bao/J. Softw.*, vol. 36, no. 8, pp. 3858–3882, 2024.
- [10] M. Miao, J. Ma, X. Huang, and Q. Wang, "Efficient verifiable databases with insertion/deletion operations from delegating polynomial functions," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 2, pp. 511–520, Feb. 2018.
- [11] S. Wu, Q. Li, G. Li, D. Yuan, X. Yuan, and C. Wang, "ServeDB: Secure, verifiable, and efficient range queries on outsourced database," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 626–637.
- [12] X. Chen et al., "Publicly verifiable databases with all efficient updating operations," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 12, pp. 3729–3740, Dec. 2021.
- [13] Q. Wang, F. Zhou, B. Zhou, J. Xu, C. Chen, and Q. Wang, "Privacy-preserving publicly verifiable databases," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1639–1654, May/Jun. 2022.
- [14] B. Fuhrig, H. A. J. Jain, and F. Kerschbaum, "EncDBDB: Searchable encrypted, fast, compressed, in-memory database using enclaves," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 438–450.
- [15] N. Crooks, M. Burke, E. Cecchetti, S. Harel, R. Agarwal, and L. Alvisi, "Obladi: Oblivious serializable transactions in the cloud," in *Proc. 13th USENIX Symp. Operating Syst. Des. Implementation*, 2018, pp. 727–743.
- [16] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, and T. Lai, "SgxPectre: Stealing intel secrets from SGX enclaves via speculative execution," *IEEE Secur. Privacy*, vol. 18, no. 03, pp. 28–37, May/Jun. 2020.
- [17] W. Li, W. Susilo, C. Xia, L. Huang, F. Guo, and T. Wang, "Secure data integrity check based on verified public key encryption with equality test for multi-cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5359–5373, Nov./Dec. 2024.
- [18] H. Yang, D. Feng, and J. Qin, "Efficient verifiable unbounded-size database from authenticated matrix commitment," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 5, pp. 3873–3889, Sep./Oct. 2023.
- [19] Y. Guan, H. Zheng, J. Shao, R. Lu, and G. Wei, "Fair outsourcing polynomial computation based on the blockchain," *IEEE Trans. Servi. Comput.*, vol. 15, no. 5, pp. 2795–2808, Sep./Oct. 2022.
- [20] Q. Wang, Y. Wang, F. Zhou, J. Xu, and C. Zhang, "Privacy-preserving fair outsourcing polynomial computation without FHE and FPR," *Comput. Standards Interfaces*, vol. 91, 2025, Art. no. 103899.
- [21] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, "One server for the price of two: Simple and fast single-server private information retrieval," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 3889–3905.
- [22] A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, and N. Zeldovich, "Private web search with tiptoe," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 396–416.
- [23] S. Angel and S. Setty, "Unobservable communication over fully untrusted infrastructure," in *Proc. 12th USENIX Symp. Operating Syst. Des. Implementation*, 2016, pp. 551–569.
- [24] R. Cheng et al., "Talek: Private group messaging with hidden access patterns," in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 84–99.
- [25] A. Ali et al., "Communication–Computation trade-offs in PIR," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1811–1828.
- [26] T. Gupta, N. Crooks, W. Mulhern, S. Setty, L. Alvisi, and M. Walfish, "Scalable and private media consumption with popcorn," in *Proc. 13th USENIX Symp. Networked Syst. Des. Implementation*, 2016, pp. 91–107.
- [27] A. Ghoshal, M. Zhou, and E. Shi, "Efficient pre-processing PIR without public-key cryptography," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2024, pp. 210–240.
- [28] D. Kogan and H. Corrigan-Gibbs, "Private blocklist lookups with checklist," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 875–892.
- [29] A. Lazzaretti and C. Papamanthou, "Single pass client-preprocessing private information retrieval," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 5967–5984.
- [30] S. Celi and A. Davidson, "Call me by my name: Simple, practical private information retrieval for keyword queries," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 4107–4121.
- [31] S. Colombo, K. Nikitin, H. Corrigan-Gibbs, D. J. Wu, and B. Ford, "Authenticated private information retrieval," in *Proc. 32nd USENIX Secur. Symp.*, Anaheim, CA, USA, Aug. 2023, pp. 3835–3851.
- [32] Z. Kaifa, Z. Ziyu, L. Jianwei, and Y. Beiyuan, "Secure fine-grained multi-keyword ciphertext search supporting cloud-edge-end collaboration in IoT," *Chin. J. Electron.*, vol. 34, no. 1, pp. 266–281, 2025. [Online]. Available: <https://cje.ejournal.org.cn/en/article/doi/10.23919/cje.2023.00.244>
- [33] Z. Xiaojun, X. Chunxiang, X. Run, and J. Chunhua, "Designated cloud server public key encryption with keyword search from lattice in the standard model," *Chin. J. Electron.*, vol. 27, no. 2, pp. 304–309, 2018. [Online]. Available: <https://cje.ejournal.org.cn/en/article/doi/10.1049/cje.2018.01.012>
- [34] D. Derler, K. Samelin, and D. Slamanig, "Bringing order to chaos: The case of collision-resistant chameleon-hashes," in *Proc. 23rd IACR Int. Conf. Pract. Theory Public-Key Cryptogr. Public-Key Cryptogr.*, Edinburgh, U.K., 2020, pp. 462–492.
- [35] T. Ye, M. Luo, Y. Yang, K.-K. R. Choo, and D. He, "A survey on redactable blockchain: Challenges and opportunities," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1669–1683, May/Jun. 2023.
- [36] J. Camenisch, D. Derler, S. Krenn, H. C. Pöhls, K. Samelin, and D. Slamanig, "Chameleon-Hashes with ephemeral trapdoors," in *Proc. Public-Key Cryptogr.*, S. Fehr, Ed., Berlin, Germany: Springer, 2017, pp. 152–182.
- [37] D. Boneh and M. Franklin, "Identity-based encryption from the weil pairing," in *Proc. Annu. Int. Cryptol. Conf.*, 2001, pp. 213–229.
- [38] D. Boneh, "The decision Diffie-Hellman problem," in *Proc. Int. Algorithmic Number Theory Symp.*, 1998, pp. 48–63.
- [39] Z. Brakerski and S. Goldwasser, "Circular and leakage resilient public-key encryption under subgroup indistinguishability: (Or: Quadratic residuosity strikes back)," in *Proc. 30th Annu. Cryptol. Conf.*, Santa Barbara, CA, USA, 2010, pp. 1–20.
- [40] C. Orlandi, P. Scholl, and S. Yakoubov, "The rise of paillier: Homomorphic secret sharing and public-key silent OT," in *Proc. 40th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Zagreb, Croatia, 2021, pp. 678–708.